



Data Handling: Import, Cleaning and Visualisation

Lecture 5: Non-rectangular data

Dr. Aurélien Sallin

2025-10-16

Recap

Goals of last lecture: from text to data

Our story:

- Computers store data as bits (Bits → Bytes → Encoded Text)
- Text encodes meaning
- Files encode structure (→ Structured Files (CSV))
- R parses structure into memory. (→ R Parsing → R Data Structures)

Goals:

- Understand how we **import** and structure data from text files
- Learn **data structures** in R
- Understand **rectangular data formats**

Using a simple parser algorithm

- **Parser**: sequence of instructions/rules of how to distinguish columns and rows when all that a computer can do is sequentially read one character after the other.
- Possible if we can agree on certain **standards** of how data structured as a table/matrix is stored in a text file.

A csv-parser takes a text file with CSV standard structure as input and builds a data structure in the form of a table.

```
00000000: efbb bf79 6561 722c 6764 705f 6368 6662 ...year,gdp_chfb
00000010: 0d31 3938 302c 3138 340d 3139 3835 2c32 .1980,184.1985,2
00000020: 3434 0d31 3939 302c 3333 310d 3139 3935 44.1990,331.1995
00000030: 2c33 3734 0d32 3030 302c 3432 320d 3230 ,374.2000,422.20
00000040: 3035 2c34 3634 05,464
```

Recap: parsing CSVs in R

- `readr`, `read_csv()`, `read_delim()` to parse csvs
 - Return a `tibble`.
- parsing, encoding, and guessing the right type: `guess_parser()`

```
read_csv('A,B
          12:00, 12:00
          14:30, midnight
          20:01, noon')
```

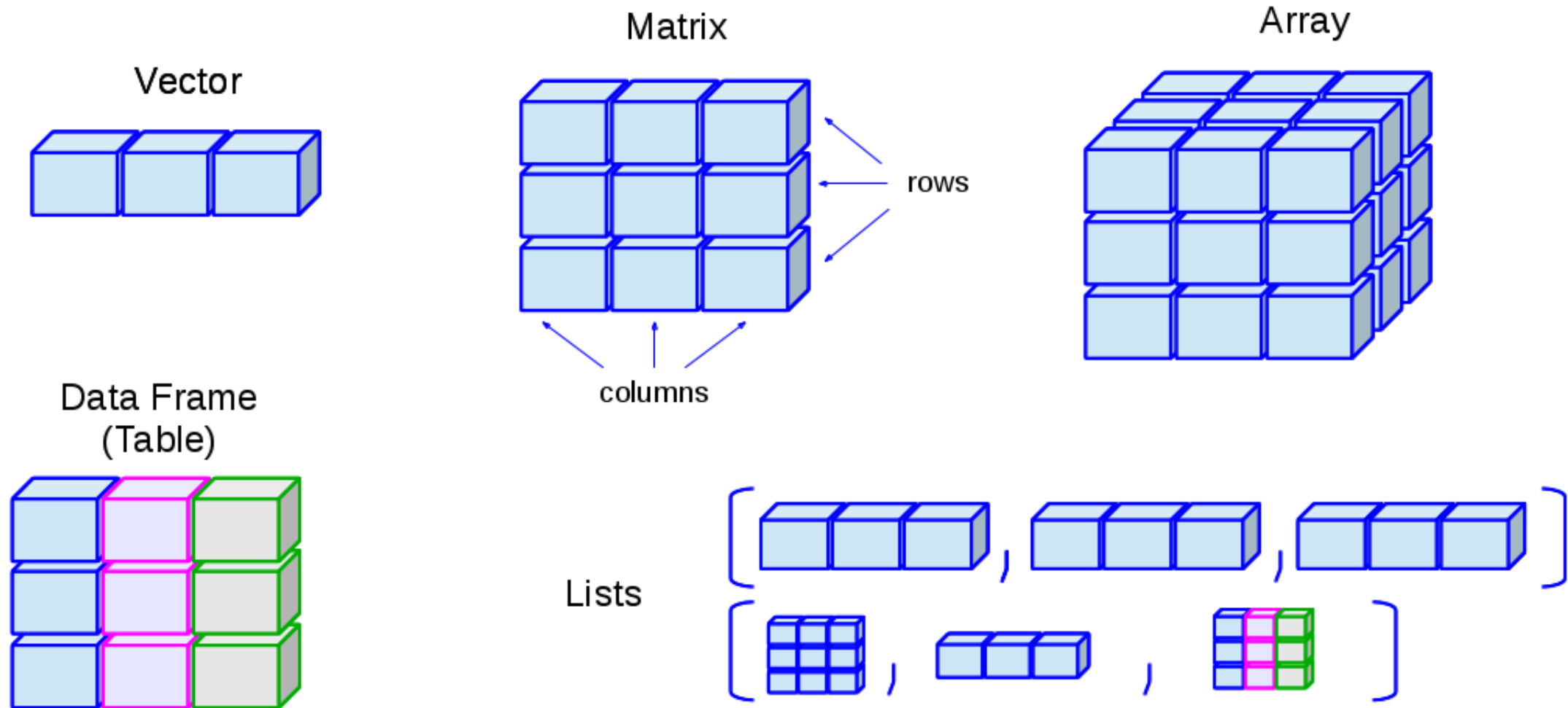
```
# A tibble: 3 × 2
  A      B
  <time> <chr>
1 12:00  12:00
2 14:30  midnight
3 20:01  noon
```

Structures to work with in R

- A data structure in R is a way of **organizing data in memory**.
- It defines how elements are grouped, stored, and accessed.
- R has a few core structures, each designed for a different kind of organization.

Structure	Key idea	Type	Dim.
Vector	Ordered collection of values	Homog.	1D
Matrix	2D rectangular collection	Homog.	2D
Array	Generalization of a matrix	Homog.	nD
List	Elements can differ in type/size	Heterog.	Flex
Data frame/Tibble	Equal-length column vectors (table)	Heterog.	2D

Different data types in one figure



Source: <http://venus.ifca.unican.es/Rintro/dataStruct.html>

Important concepts

coercion

```
persons <- c("Frodo", "Gandalf", "Sam")
ages <- c(24, 5000, 28)
c(persons, ages)
```

```
[1] "Frodo" "Gandalf" "Sam" "24" "5000" "28"
```

homogenous and heterogenous types

```
matrix(c(1, 2, 3, "Frodo", "Gandalf", "Sam"), ncol = 2)
```

```
  [,1] [,2]
[1,] "1" "Frodo"
[2,] "2" "Gandalf"
[3,] "3" "Sam"
```

```
data.frame("ID" = c(1, 2, 3), "NAME" = c("Frodo", "Gandalf", "Sam"))
```

```
  ID  NAME
1  1 Frodo
2  2 Gandalf
3  3 Sam
```

Important concepts

factors

```
middle_earth <- factor(c("Hobbit", "Dwarf", "Elf", "Dwarf", "Elf", "Hobbit"),  
                      levels = c("Hobbit", "Dwarf", "Elf"))  
  
middle_earth <- factor(c(1,2,3,2,3,1),  
                      labels = c("Hobbit", "Dwarf", "Elf"))  
  
sort(middle_earth)
```

```
[1] Hobbit Hobbit Dwarf  Dwarf  Elf    Elf  
Levels: Hobbit Dwarf Elf
```

```
typeof(middle_earth)
```

```
[1] "integer"
```

Work with data.frames

Select columns

```
swiss$Fertility # use the $-operator  
swiss[, 1] # use brackets [] and the column number/index  
swiss[, "Fertility"] # use the name of the column  
swiss[, c("Fertility", "Agriculture")] # use the name of the column
```

Select rows

```
swiss[1,] # First row  
swiss[swiss$Fertility > 40,] # Based on condition ("filter")
```

Create columns

```
swiss$Protestant <- 100 - swiss$Catholic
```

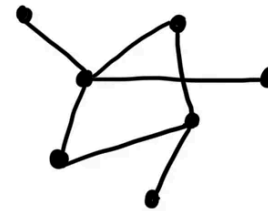
Data in Economics

Data take different structures depending on their purpose.

RECTANGULAR

id	name	number	has_pet
1	Amy	123-4567	1
2	Bob	246-8101	0
3	Carl	135-7911	0
4	Dave	765-4321	1

NON-RECTANGULAR



```
{ "users": [
  {
    "name": "Jasmin",
    "membersince": "2024"
  },
  ...
]
```



Recap: exercise

Read financial data with another separator and with encoding problems.

```
financial_data <- read.csv("financial_data.txt", sep = ":")  
financial_data <- read_delim("financial_data.txt", delim = ":")
```

- Error with `read.csv()`: “invalid multibyte string at ”.
- No error with `read_delim()`: more robust to encoding issues!

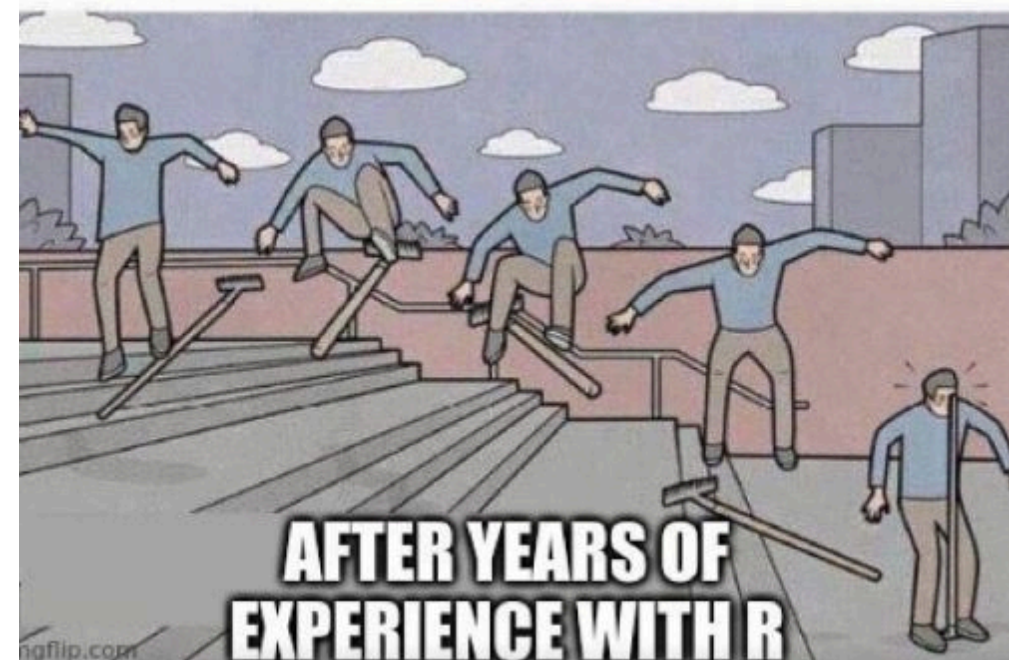
Recap: exercise

With `iconv()`, we found the right encoding for `\xF6`. We could then import the data using

```
iconv("\xF6", from = "ISO-8859-1", to = "UTF-8")
financial_data <- read.csv("financial_data.txt",
                          sep = ":",
                          fileEncoding = "ISO-8859-1")
```

However, because of the special character, the column was read as a character. We replaced the value and coerced the whole column to numeric.

```
financial_data[10, "Revenue"] <- 1933
financial_data[, "Revenue"] <- as.numeric(financial_data[, "Revenue"])
```



Warm-up

Coercion of boolean values

This question checks your understanding of the coercion of boolean values.

What is the output of the following code?

```
my_vector <- c(1, 0, 3, -1)
as.numeric(my_vector > 0)
```

- "TRUE" "FALSE" "TRUE" "FALSE"
- TRUE FALSE TRUE FALSE
- 1 0 1 0
- 0 1 0 1

Data frames operations

Consider the data frame

```
dataCHframe <- data.frame(  
  "City" = c("St.Gallen", "Lausanne", "Zürich"),  
  "PartyLeft" = c(35, 45, 55),  
  "PartyRight" = c(40, 35, 30)  
)
```

Which of these statements are TRUE?

- `dataCHframe$PartyCenter <- c(25, 20, 15)` creates a new variable called "PartyCenter"
-   `dim(dataCHframe[, dataCHframe$PartyLeft > 40])` returns the same as `dim(dataCHframe[, c(2,3)])`
- `dim(dataCHframe[dataCHframe$PartyLeft > 40 | dataCHframe$PartyLeft < 40, ])` returns `c(3,3)`
- `dataCHframe` is a data.frame, which is a list consisting of one named character vector and two named integer vectors.

Data frames

Consider the following code

```
df <- data.frame(a = c(1,2,3,4),  
                 b = c("au", "de", "ch", "li"))
```

Are these statements TRUE or FALSE?

- `mean(df$a) == 2.5`
- `typeof(as.matrix(df)[,1])` is `numeric` (or `double`)

Updates

Next week

- “Leverage APIs for Economic Data” with Minna Heim
- ⚡ provides with the basis knowledge to solve the first part of the group project

Today

Goals of today's lecture

- Understand **non-rectangular data**: xml, json, and html
- Be familiar with the way we **parse** these data into R
- **Scrape a webpage** following its html structure

Non-rectangular data structures

Non-rectangular data

- Hierarchical data (xml, html, json)
 - XML and JSON (useful for complex/high-dimensional data sets).
 - HTML (a markup language to define the structure and layout of webpages).
- Unstructured text data
- Images/Pictures data

A rectangular data set

father	mother	name	age	gender
		John	33	male
		Julia	32	female
John	Julia	Jack	6	male
John	Julia	Jill	4	female
John	Julia	John jnr	2	male
		David	45	male
		Debbie	42	female
David	Debbie	Donald	16	male
David	Debbie	Dianne	12	female

 What is the data about?

A rectangular data set

```
father mother name age gender
John      John  John   33  male
John      John  Julia  32  female
John      Julia Jack    6  male
John      Julia Jill    4  female
John      Julia John jnr 2  male
          David 45  male
          Debbie 42 female
David     Debbie Donald 16  male
David     Debbie Dianne 12  female
```

Which observations belong together?

A rectangular data set

```
father mother name age gender
John      John  John  33  male
John      John  Julia 32  female
John      John  Jack   6  male
John      John  Jill   4  female
John      John  John jnr 2  male
          John  David  45  male
          John  Debbie 42  female
David     Debbie Donald 16  male
David     Debbie Dianne 12  female
```

Can a parser understand which observations belong together?

Deciphering XML

COVID-19 data in XML format

What features does the format have? What is its logic/syntax?

```
<?xml version="1.0" encoding="UTF-8"?>
<data>
  <row>
    <unique_id>216498</unique_id>
    <indicator_id>386</indicator_id>
    <name>Ozone (O3)</name>
    <measure>Mean</measure>
    <measure_info>ppb</measure_info>
    <geo_type_name>CD</geo_type_name>
    <geo_join_id>313</geo_join_id>
    <geo_place_name>Coney Island (CD13)</geo_place_name>
    <time_period>Summer 2013</time_period>
    <start_date>2013-06-01T00:00:00</start_date>
    <data_value>34.64</data_value>
  </row>

  <row>
    <unique_id>216499</unique_id>
    <indicator_id>386</indicator_id>
    ...
  </row>
</data>
```

COVID-19 data in XML format

What features does the format have? What is its logic/syntax?

```
<?xml version="1.0" encoding="UTF-8"?>
<data>
  <row>
    <unique_id>216498</unique_id>
    <indicator_id>386</indicator_id>
    <name>Ozone (O3)</name>
    <measure>Mean</measure>
    <measure_info>ppb</measure_info>
    <geo_type_name>CD</geo_type_name>
    <geo_join_id>313</geo_join_id>
    <geo_place_name>Coney Island (CD13)</geo_place_name>
    <time_period>Summer 2013</time_period>
    <start_date>2013-06-01T00:00:00</start_date>
    <data_value>34.64</data_value>
  </row>

  <row>
    <unique_id>216499</unique_id>
    <indicator_id>386</indicator_id>
    ...
  </row>
</data>
```

- A predefined set of **special characters** (here primarily `<`, `>`, and `/`) give the data structure.
- So-called **XML-tags** are used to define variable names and encapsulate data values:
`<variablename>value</variablename>`.
- Tags can be nested, which allows for the definition of hierarchical structures.

XML declaration

- An XML document begins with some information about XML itself.
- For example, it might mention the XML version that it follows.
- This opening is called an **XML declaration**.

```
<?xml version="1.0" encoding="UTF-8"?>
```

XML markup is structured by special characters and tags

The “row-content” is **nested** between the ‘row’-tags:

```
<row>  
...  
</row>
```

XML markup is structured by special characters and tags

A xml element:

```
<geo_place_name>Coney Island (CD13)</geo_place_name>
```

- start tag: `<geo_place_name>Coney Island (CD13)</geo_place_name>`
- data value: `<geo_place_name>Coney Island (CD13)</geo_place_name>`
- end tag: `<geo_place_name>Coney Island (CD13)</geo_place_name>`

Originally, XML element values were limited to a single type: strings. It is however possible to write a schema that assigns data types to the elements of a XML document (referred to as XSD or XML Schema Definition). See [Mundell](#), chapter 5.5.3 and 5.54 for more details.

XML markup has a nested syntax... similar to a family-tree structure 🌳

Tags can be nested, which allows for the definition of hierarchical structures.

Here, the “root-content” is nested between the ‘root’-tags:

```
<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>
```

XML syntax: Temperature Data example

There are two principal ways to link variable names to values.

```
<?xml version="1.0" encoding="UTF-8"?>
<dataset>
  <variable>Monthly Surface Clear-sky Temperature (ISCCP) (Celsius)</variable>
  <filename>ISCCPMonthly_avg.nc</filename>
  <filepath>/usr/local/fer_data/data/</filepath><|meta_end|>filepath>
  <badflag>-1.E+34</badflag>
  <subset>48 points (TIME)</subset>
  <longitude>123.8W(-123.8)</longitude>
  <latitude>48.8S</latitude>
  <case date="16-JAN-1994" temperature="9.200012" />
  <case date="16-FEB-1994" temperature="10.70001" />
  <case date="16-MAR-1994" temperature="7.5" />
  <case date="16-APR-1994" temperature="8.100006" />
</dataset>
```

XML syntax

There are two principal ways to link variable names to values in XML

Tag/element-defined

Define opening and closing XML-tags with the variable name and surround the value with them:

```
<filename>ISCCPMonthly_avg.nc</filename>
```

Attribute-defined

Encapsulate the values within one tag by defining tag-attributes:

```
<case date="16-JAN-1994" temperature="9.200012" />
```

There are two principal ways to link variable names to values in XML

Attribute-defined

```
<case date="16-JAN-1994" temperature="9.200012" />
<case date="16-FEB-1994" temperature="10.70001" />
<case date="16-MAR-1994" temperature="7.5" />
<case date="16-APR-1994" temperature="8.100006" />
```

Tag/element-defined

```
<cases>
  <case>
    <date>16-JAN-1994</date>
    <temperature>9.200012</temperature>
  </case>
  <case>
    <date>16-FEB-1994</date>
    <temperature>10.70001</temperature>
  </case>
  <case>
    <date>16-MAR-1994</date>
    <temperature>7.5</temperature>
  </case>
  <case>
    <date>16-APR-1994</date>
    <temperature>8.100006</temperature>
  </case>
</cases>
```

XML - eXtensible Markup Language

- Since 1998
- Real life examples:
 - Microsoft Office uses the XML-based file formats, such as .docx, .xlsx, and .pptx.
 - Store application configuration data
 - Store geospatial data in Geographic Information Systems (GIS)
 - ...

XML representation of a Word document

- ▶ See a XML representation of a Word document on [the Wikipedia page on Microsoft Office XML format](#)

XML in a nutshell

- **Human-readable:** XML files can be easily read and understood without needing special tools.
- **Self-descriptive:** XML tags describe the data they enclose.
- **Hierarchical structure:** Data is organized in a tree-like structure, making it easy to represent complex relationships.

Advantages of XML over csv

- **More complex (multi-dimensional)** in XML-files than what is possible in CSVs.
- **Self-explanatory syntax:** machine-readable and human-readable.
- **Structured:** give both structure and name variables.

Potential drawback of XML: inefficient storage

- Tags are part of the syntax, thus, part of the actual file.
 - Tags (variable labels) are repeated again and again!
 - CSV: variable labels are mentioned once.
- Potential solution: **data compression** (e.g., zip).
- If the data is actually two-dimensional, a CSV is more practical.

Parsing XML

XML in R with *The Office*

```
<?xml version="1.0" encoding="UTF-8"?>
<company_dundermifflin>
  <person id="1">
    <name>Michael Scott</name>
    <position>Regional Manager</position>
    <location branch="Scranton"/>
  </person>
  <person id="2">
    <name>Dwight Schrutte</name>
    <position>Assistant (to the) Regional Manager</position>
    <location branch="Scranton"/>
    <orders>
      <sales>
        <units>10</units>
        <product>paper A4</product>
      </sales>
    </orders>
  </person>
  <person id="3">
    <name>Jim Halpert</name>
    <position>Sales Representative</position>
    <location branch="Scranton"/>
    <orders>
```

- The root is `company_dundermifflin`
- `company_dundermifflin` is the root-node, `persons` are its children
- Three persons: Michael Scott, Dwight Schrutte, Jim Halpert
- For Dwight and Jim, we have paper sales orders.

Load a XML file in R

```
# load packages
library(xml2) # updated and faster than library `XML`

# parse XML, represent XML document as R object
xml_doc <- read_xml("company_dundermifflin.xml")
xml_doc
```

```
{xml_document}
<company_dundermifflin>
[1] <person id="1">\n  <name>Michael Scott</name>\n  <position>Regional Manager</position>\n  <lo ...
[2] <person id="2">\n  <name>Dwight Schrutte</name>\n  <position>Assistant (to the) Regional Mana ...
[3] <person id="3">\n  <name>Jim Halpert</name>\n  <position>Sales Representative</position>\n  < ...
```

Identify the root-node and the children

▼ Navigate downwards (find the children or the first child)

```
xml_children(xml_doc)
xml_child(xml_children(xml_doc), 1)
persons <- xml_children(xml_doc)
```

► Show output

◀ ▶ Navigate sideways

```
xml_siblings(persons[1])
```

► Show output

▲ Navigate upwards: find all the parents (all levels) or the parent (one level)

```
sales <- xml_children(xml_children(xml_children(xml_doc)))
xml_parents(sales)
xml_parent(sales)
```

► Show output

Extract specific parts of the data using XPath

XPath is a language specifically designed for navigating and querying XML documents

```
# find data via XPath
sales <- xml_find_all(xml_doc, xpath = ".//sales")
sales
```

```
{xml_node_set (3)}
[1] <sales>\n  <units>10</units>\n  <product>paper A4</product>\n</sales>
[2] <sales>\n  <units>20</units>\n  <product>paper A4</product>\n</sales>
[3] <sales>\n  <units>5</units>\n  <product>paper A3</product>\n</sales>
```

```
xml_text(sales) # extract the data as text
```

```
[1] "10paper A4" "20paper A4" "5paper A3"
```

```
position <- xml_find_all(xml_doc, xpath = ".//position")
position
```

```
{xml_node_set (3)}
[1] <position>Regional Manager</position>
[2] <position>Assistant (to the) Regional Manager</position>
[3] <position>Sales Representative</position>
```

Deciphering JSON

JSON = JavaScript Object Notation

- **Text-based format** used for representing structured and based on JavaScript object syntax.
- Used for data exchange between a server and a web application
- Used in most APIs -> see next lecture.

JSON syntax

- Key difference to XML: no tags, but **attribute-value pairs**.
- JSON is built on two structures:
 - **Object**: a collection of properties, i.e. key/value pairs `{"key": value}`
 - **Array**: an ordered list of values `[1, 2, 3]`
- A substitute for XML (often encountered in similar usage domains).

Object properties are accessed using keys. Keys must be strings, and values must be a valid JSON data type (string, number, object, array, boolean or null). Arrays can contain objects.

XML:

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumber>
    <type>home</type>
    <number>212 555-1234</number>
  </phoneNumber>
  <phoneNumber>
    <type>fax</type>
    <number>646 555-4567</number>
  </phoneNumber>
  <gender>
    <type>male</type>
  </gender>
</person>
```

JSON:

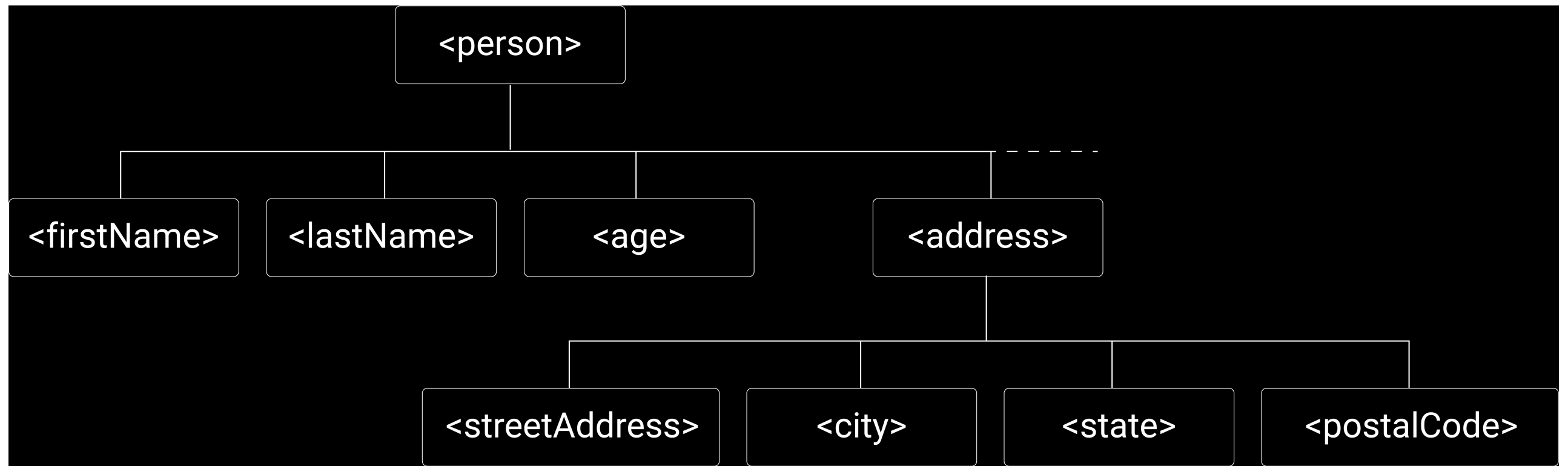
```
{ "firstName": "John",
  "lastName": "Smith",
  "age": 25,
  "address": {
    "streetAddress": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postalCode": "10021"
  },
  "phoneNumber": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "fax",
      "number": "646 555-4567"
    }
  ],
  "gender": {
    "type": "male"
  }
}
```

XML:

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
</person>
```

JSON:

```
{
  "firstName": "John",
  "lastName": "Smith",
}
```



Parsing JSON

JSON in R

```
# load packages
library(jsonlite)

# parse the JSON-document shown in the example above
json_doc <- fromJSON("data/person.json")

# look at the structure of the document
str(json_doc)
```

```
List of 6
 $ firstName  : chr "John"
 $ lastName   : chr "Smith"
 $ age        : int 25
 $ address    :List of 4
  ..$ streetAddress: chr "21 2nd Street"
  ..$ city          : chr "New York"
  ..$ state         : chr "NY"
  ..$ postalCode    : chr "10021"
 $ phoneNumber:'data.frame':  2 obs. of  2 variables:
  ..$ type  : chr [1:2] "home" "fax"
  ..$ number: chr [1:2] "212 555-1234" "646 555-4567"
 $ gender     :List of 1
  ..$ type: chr "male"
```

JSON in R

The nesting structure is represented as a *nested list*:

```
# navigate the nested lists, extract data  
# extract the address part  
json_doc$address
```

```
$streetAddress  
[1] "21 2nd Street"  
  
$city  
[1] "New York"  
  
$state  
[1] "NY"  
  
$postalCode  
[1] "10021"
```

```
# extract the gender (type)  
json_doc$gender$type
```

```
[1] "male"
```

HTML: Websites

HTML: Code to build webpages

HyperText Markup Language (HTML), designed to be read by a web browser.

It is a markup language, not a programming language, used to structure the content of webpages.



HTML documents: code and data!

HTML documents/webpages consist of '*semi-structured data*':

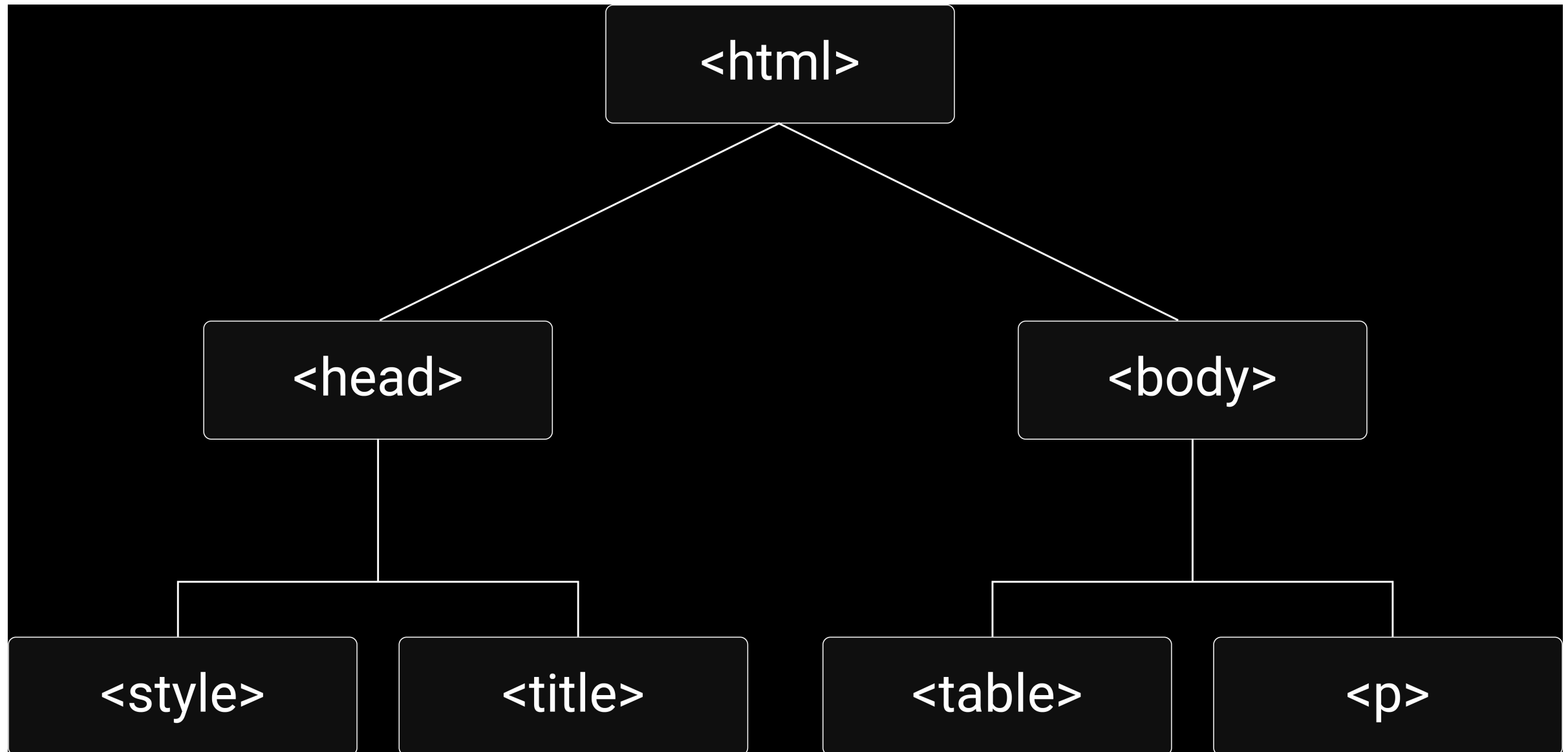
- A webpage can contain a HTML-table (*structured data*)...
- ...but likely also contains just raw text (*unstructured data*).

```
<!DOCTYPE html> <!-- Defines this document as HTML5 -->

<html>
  <head>
    <title>hello, world</title>
  </head>
  <body>
    <h2> hello, world </h2>
  </body>
</html>
```

Similarities to other formats?

HTML document as a 'tree'



Read a webpage into R

In this example, we look at [Wikipedia's Economy of Switzerland page](https://en.wikipedia.org/wiki/Economy_of_Switzerland).

Year	GDP (billions of CHF)	US Dollar Exchange
1980	184	1.67 Francs
1985	244	2.43 Francs
1990	331	1.38 Francs
1995	374	1.18 Francs
2000	422	1.68 Francs
2005	464	1.24 Francs
2006	491	1.25 Francs
2007	521	1.20 Francs
2008	547	1.08 Francs
2009	535	1.09 Francs
2010	546	1.04 Francs
2011	659	0.89 Francs
2012	632	0.94 Francs
2013	635	0.93 Francs
2014	644	0.92 Francs
2015	646	0.96 Francs
2016	659	0.98 Francs
2017	668	1.01 Francs
2018	694	1.00 Francs

Source: https://en.wikipedia.org/wiki/Economy_of_Switzerland

Tutorial (advanced): Importing data from a HTML table

-> Exercise session last week!

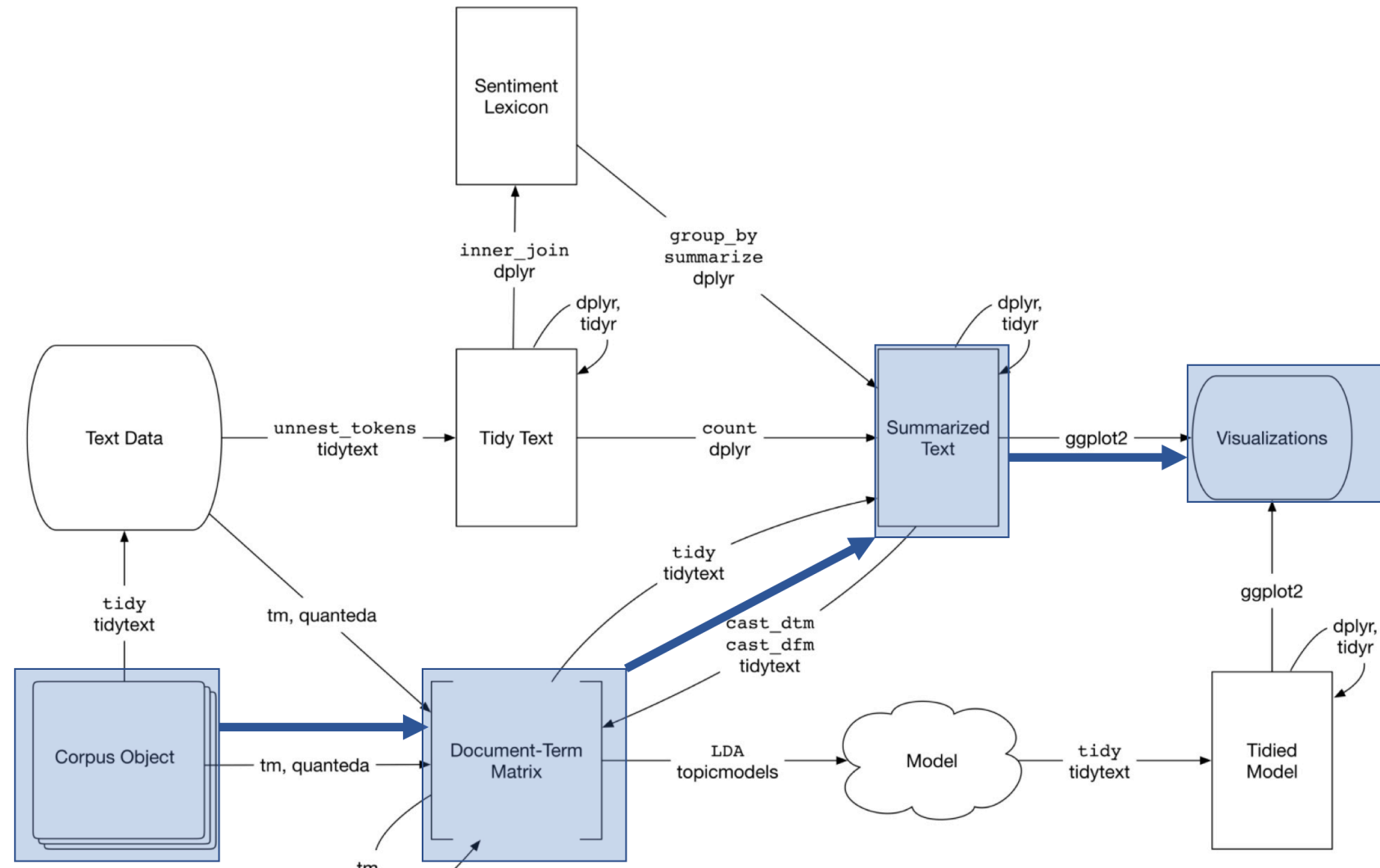
Text as Data and Image Data

Text as Data became more common as data source

- Extract text from historical documents (scan, use OCR)
- Use machine learning to label text (too costly to do manually)
- Extract sentiment (finance for instance)

Text is unstructured data. Text analysis and feature extraction is the basis for new genAI models!

An example of text mining in R



Source: <https://confessionsofadatascientist.com/text-analysis-with-r-repro-report.html>

Self-study on text mining

Read Chapter 6 on text data in [the book by U. Matter](#)

Key concepts:

- **regular expressions**: flexible syntax used to detect and describe patterns in strings
- **corpus**: a collection of authentic text organized into datasets
- **documents**
- **tokens and N-grams**
- **document-term-matrix (dtm)**

Key libraries:

- **quanteda**: see [Quanteda](#)
- **tidytext**: see [Text Mining with R](#)

Happy break!!!

